PHP 6 was abandoned not least because of the difficulties in implementing full support for Unicode, which is a vast collection of collation rules in addition to being a vast collection of characters (as we'll see in the next section). But the syntax for binary strings still exists and has not been deprecated (yet), and therefore, the following code is perfectly legal even in PHP 8.4:

```
var_dump("a string" === b'a string');

$str = <<<STR
a string
STR;

var_dump($str === b'a string');

Output:
bool(true)
bool(true)
```

Here we've used a type-strict === comparison, and in both cases the `b'';` form is identical to the other strings. All strings in PHP are "binary strings" – a sequence of bytes with no special significance to PHP relative to any particular character encoding – interpretation is left to the client. Note that the phrase "binary string" is sometimes encountered when describing a string composed only of the digit characters "1" and "0," used to represent a binary number, such as you might encounter with the function `bindec()` (see Chapter 7, "Numbers," for more detail), but generally "binary string" is usually synonymous with "byte stream" or "buffer," which we'll cover later.

The development effort around PHP 6 and full Unicode support began to outstrip the value it was adding to the language, and so PHP 6 was shelved, PHP 7 became the successor to PHP 5, and Unicode support was instead added to the language via functions and built-in classes. Here endeth the lesson. If phrases such as "Unicode" and "multi-byte characters" are unfamiliar to you, fear not: we'll see all of this in detail throughout the rest of the chapter.

(Footnote: it can be argued that PHP 5.3 represented the true release of PHP 6 because it added so many new things to the language, if you're the arguing type…)